

새싹 강동캠퍼스 베스핀글로벌

AWS VPC 3-Tier + Bastion Host 구축 수행평가

미션 가이드 · 구축 절차 · 통신 검증 · 평가 시트

WEB → WAS → DB

Nginx :80 → Flask :5000 → MySQL :3306

강사 젠아이랩스 박수현

새싹 강동캠퍼스 · 베스핀글로벌

수행평가 미션 및 평가 시트

수행평가 목표

AWS VPC 환경에서 **WEB / WAS / DB 3-Tier 아키텍처**를 직접 설계하고 구축한다.

2 개의 Availability Zone, Public/Private Subnet, Zonal NAT Gateway, Route Table, Bastion Host, Security Group 을 구성하고

최종적으로 **Nginx → Flask → MySQL** 로 이어지는 서비스 통신과 계층별 접근 통제를 검증한다.

시나리오

A 사는 온라인으로 상품 정보를 제공하는 신규 웹 서비스를 AWS 환경에 구축하려고 합니다. 서비스는 외부 사용자가 접속하는 WEB 계층, 비즈니스 로직을 처리하는 WAS 계층, 상품 정보를 저장하는 DB 계층으로 분리하여 구성해야 합니다.

서비스의 안정성을 높이기 위해 2 개의 Availability Zone 을 활용한 Multi-AZ 구조로 설계하고, WEB 서버만 인터넷에서 직접 접근할 수 있도록 Public Subnet 에 배치합니다. WAS 와 DB 서버는 외부에서 직접 접근할 수 없는 Private Subnet 에 배치하며, 관리자는 Bastion Host 를 통해서만 Private 서버에 접속할 수 있어야 합니다.

또한 각 서버 간에는 업무에 필요한 최소한의 통신만 허용해야 합니다. WEB 서버는 WAS 서버의 애플리케이션 포트만 접근하고, WAS 서버는 DB 서버의 MySQL 포트만 접근하도록 구성합니다. WEB 서버가 DB 서버에 직접 접근하거나 외부 인터넷에서 WAS 또는 DB 서버에 직접 접근하는 것은 차단되어야 합니다.

여러분은 A 사의 클라우드 인프라 담당자로서 위 요구사항을 만족하는 AWS VPC 기반 3-Tier 아키텍처를 설계·구축하고, 각 계층 간 허용 및 차단 정책이 정상적으로 동작하는지 검증해야 합니다.

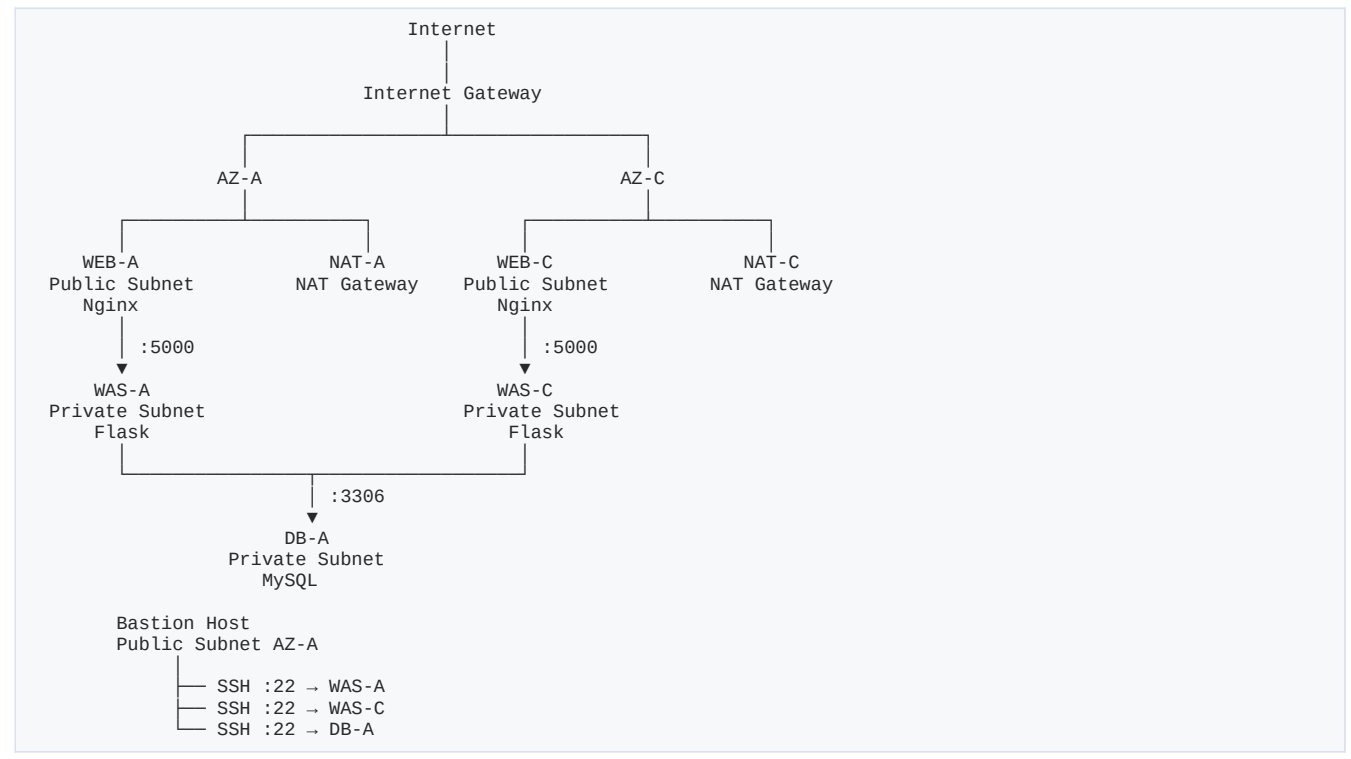
1. Mission

다음 조건을 모두 만족하는 AWS 인프라를 구축한다.

필수 구성

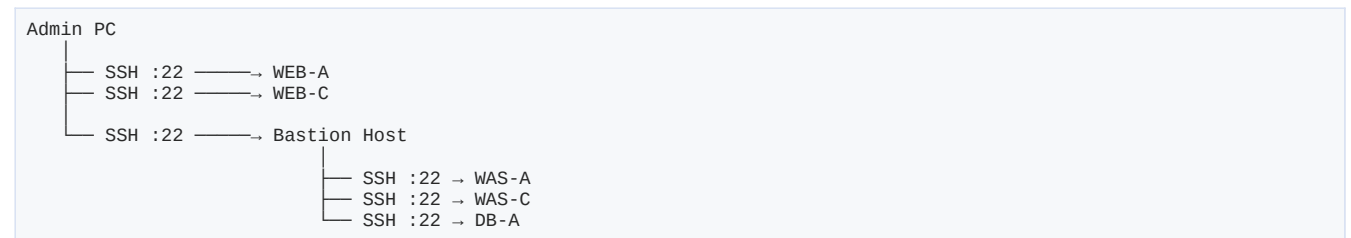
- VPC 1 개
- Availability Zone 2 개 사용
- WEB / WAS / DB 3-Tier 구성
- Subnet 총 6 개
- Internet Gateway 1 개
- Zonal NAT Gateway 2 개
- Route Table 3 개
- Bastion Host 1 대
- WEB EC2 2 대
- WAS EC2 2 대
- DB EC2 1 대
- WEB 서버: Nginx
- WAS 서버: Python Flask
- DB 서버: MySQL
- WEB → WAS → DB 서비스 연동
- Security Group 을 이용한 최소 포트 접근 통제
- 허용되어야 하는 통신과 차단되어야 하는 통신을 직접 검증

2. 최종 아키텍처



3. 접근 정책

관리 접속

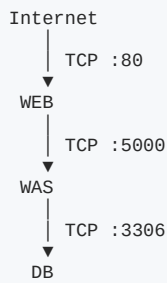


반드시 지켜야 할 조건

- WEB-A / WEB-C 는 Public IP 를 가진다.
- Bastion Host 는 Public IP 를 가진다.
- WAS-A / WAS-C / DB-A 는 **Public IP 를 가지지 않는다.**
- Private Subnet 의 서버는 인터넷에서 직접 SSH 접속할 수 없어야 한다.
- Private 서버 SSH 는 반드시 Bastion Host 를 경유한다.

4. 서비스 통신 정책

서비스 트래픽은 다음 방향만 허용한다.



허용

Internet	→	WEB	:	80
WEB	→	WAS	:	5000
WAS	→	DB	:	3306
관리자	→	WEB	:	22
관리자	→	Bastion	:	22
Bastion	→	WAS	:	22
Bastion	→	DB	:	22

차단

다음 통신은 성공하면 안 된다.

Internet	→	WAS	:	5000	X
Internet	→	DB	:	3306	X
WEB	→	DB	:	3306	X
WEB	→	WAS	:	3306	X
WAS	→	DB	:	5000	X
관리자	→	WAS	:	22	X
관리자	→	DB	:	22	X

5. VPC 주소 설계

VPC

10.0.0.0/16

6. Subnet 주소 설계

Tier	AZ-A	AZ-C	유형
WEB	10.0.10.0/24	10.0.20.0/24	Public
WAS	10.0.11.0/24	10.0.21.0/24	Private
DB	10.0.12.0/24	10.0.22.0/24	Private

주소 규칙:

10.0.1x.0/24 → AZ-A
10.0.2x.0/24 → AZ-C

x0 → WEB
x1 → WAS
x2 → DB

7. EC2 배치

서버	Subnet	Public IP	역할
Bastion	WEB-A Public Subnet	O	Private 서버 관리 접속
WEB-A	WEB-A Public Subnet	O	Nginx
WEB-C	WEB-C Public Subnet	O	Nginx
WAS-A	WAS-A Private Subnet	X	Flask
WAS-C	WAS-C Private Subnet	X	Flask
DB-A	DB-A Private Subnet	X	MySQL

DB-C Subnet 은 Multi-AZ 확장용으로 생성하지만 이번 평가에서는 DB EC2 를 생성하지 않아도 된다.

8. EC2 Instance Type

모든 EC2 는 다음 중 하나를 선택하여 생성한다.

```
t3.micro
t3.small
t3.medium
```

학생은 서버의 역할과 필요 자원을 고려하여 적절한 Instance Type 을 선택한다.

예시:

서버	선택 예
Bastion	t3.micro
WEB	t3.micro
WAS	t3.micro 또는 t3.small
DB	t3.small 또는 t3.medium

위 표는 예시이며 반드시 동일하게 구성할 필요는 없다.

9. Internet Gateway

Internet Gateway 를 생성하여 VPC 에 연결한다.

```
Internet
  ↓
Internet Gateway
  ↓
VPC 10.0.0.0/16
```

10. NAT Gateway

Zonal NAT Gateway 를 각 AZ 에 하나씩 생성한다.

NAT Gateway	Subnet	Elastic IP
NAT-GW-A	WEB-A Public Subnet	할당
NAT-GW-C	WEB-C Public Subnet	할당

Private Subnet 은 같은 AZ 의 NAT Gateway 를 사용한다.

```
WAS-A / DB-A → NAT-GW-A
WAS-C / DB-C → NAT-GW-C
```

11. Route Table

총 3개의 Route Table 을 구성한다.

11.1 RT-PUBLIC

연결 Subnet:

WEB-A
WEB-C

Route:

10.0.0.0/16 → local
0.0.0.0/0 → Internet Gateway

Bastion Host 도 WEB-A Public Subnet 에 위치하므로 RT-PUBLIC 을 사용한다.

11.2 RT-PRIVATE-A

연결 Subnet:

WAS-A
DB-A

Route:

10.0.0.0/16 → local
0.0.0.0/0 → NAT-GW-A

11.3 RT-PRIVATE-C

연결 Subnet:

WAS-C
DB-C

Route:

10.0.0.0/16 → local
0.0.0.0/0 → NAT-GW-C

12. Security Group 전체 설계

다음 4 개의 Security Group 을 생성한다.

```
SG-BASTION
SG-WEB
SG-WAS
SG-DB
```

13. SG-BASTION

Bastion Host 전용 Security Group.

Inbound

Protocol	Port	Source	목적
TCP	22	관리자 공인 IP /32	관리자 SSH

예:

```
203.0.113.10/32 → TCP 22
```

0.0.0.0/0 로 SSH 를 열지 않는 것을 권장한다.

Outbound

수행평가에서는 기본값인 All Traffic 허용을 사용해도 된다.

14. SG-WEB

WEB-A / WEB-C 에 적용한다.

Inbound

Protocol	Port	Source	목적
TCP	80	0.0.0.0/0	웹 서비스
TCP	22	관리자 공인 IP /32	WEB 직접 관리

WEB 서버는 평가 조건상 관리자 PC 에서 직접 SSH 접속을 허용한다.

반드시 없어야 하는 Rule

```
TCP 3306 → SG-WEB
```

WEB 은 MySQL 에 직접 접속하지 않는다.

15. SG-WAS

WAS-A / WAS-C 에 적용한다.

Inbound

Protocol	Port	Source	목적
TCP	5000	SG-WEB	Nginx → Flask
TCP	22	SG-BASTION	Bastion → WAS SSH

핵심

Flask 5000 번 포트의 Source 를 다음처럼 설정하면 안 된다.

```
0.0.0.0/0
10.0.0.0/16
```

반드시 가능한 경우 SG-WEB 을 Source Security Group 으로 지정한다.

16. SG-DB

DB-A 에 적용한다.

Inbound

Protocol	Port	Source	목적
TCP	3306	SG-WAS	Flask → MySQL
TCP	22	SG-BASTION	Bastion → DB SSH

반드시 없어야 하는 Rule

다음과 같은 Rule 은 추가하지 않는다.

```
3306 ← SG-WEB
3306 ← 0.0.0.0/0
3306 ← 10.0.0.0/16
```

MySQL 은 WAS 계층에서만 접근 가능해야 한다.

17. Security Group 통신 Matrix

Source	Destination	Port	결과
Internet	WEB	80	✓ 허용
관리자 PC	WEB	22	✓ 허용
관리자 PC	Bastion	22	✓ 허용
Bastion	WAS	22	✓ 허용
Bastion	DB	22	✓ 허용
WEB	WAS	5000	✓ 허용
WAS	DB	3306	✓ 허용
WEB	DB	3306	✗ 차단
Internet	WAS	5000	✗ 차단
Internet	DB	3306	✗ 차단
관리자 PC	WAS	22	✗ 직접 접속 불가
관리자 PC	DB	22	✗ 직접 접속 불가

18. Bastion Host 접속

Bastion Host 는 Public IP 를 사용한다.

```
ssh -i key.pem ubuntu@BASTION_PUBLIC_IP
```

Private EC2 는 Public IP 가 없으므로 직접 인터넷에서 접속할 수 없다.

19. ProxyJump 를 이용한 Private Server 접속

관리자 PC 에서 다음과 같이 Bastion 을 경유하여 접속할 수 있다.

WAS-A

```
ssh -i key.pem \  
-J ubuntu@BASTION_PUBLIC_IP \  
ubuntu@WAS_A_PRIVATE_IP
```

WAS-C

```
ssh -i key.pem \  
-J ubuntu@BASTION_PUBLIC_IP \  
ubuntu@WAS_C_PRIVATE_IP
```

DB-A

```
ssh -i key.pem \  
-J ubuntu@BASTION_PUBLIC_IP \  
ubuntu@DB_A_PRIVATE_IP
```

동일 Key Pair 를 사용하는 실습 환경을 기준으로 한다.

20. WEB Server - Nginx

WEB-A / WEB-C 에서 실행한다.

```
sudo apt update
sudo apt install -y nginx
```

확인:

```
systemctl status nginx
```

21. WEB-A Reverse Proxy

```
sudo vi /etc/nginx/sites-available/default
server {
    listen 80;

    location / {
        proxy_pass http://10.0.11.10:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

10.0.11.10 대신 실제 WAS-A Private IP 를 입력한다.

적용:

```
sudo nginx -t
sudo systemctl restart nginx
```

22. WEB-C Reverse Proxy

```
server {
    listen 80;

    location / {
        proxy_pass http://10.0.21.10:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

10.0.21.10 대신 실제 WAS-C Private IP 를 입력한다.

23. WAS Server - Python / Flask

WAS-A / WAS-C 에서 실행한다.

```
sudo apt update
sudo apt install -y python3 python3-pip python3-venv
```

가상환경 생성:

```
python3 -m venv venv
source venv/bin/activate
```

Python Package 설치:

```
pip install flask pymysql
```

24. Flask Application

app.py

```
from flask import Flask
import pymysql

app = Flask(__name__)

# [개선] X-Real-IP를 Flask 기본 Access Log의 Client IP로 사용
# from werkzeug.serving import WSGIRequestHandler
#
# class RealIPHandler(WSGIRequestHandler):
#     def address_string(self):
#         return self.headers.get("X-Real-IP") or super().address_string()

@app.route("/")
def index():
    conn = pymysql.connect(
        host="10.0.12.10",
        user="webuser",
        password="password",
        database="testdb"
    )

    cursor = conn.cursor()
    cursor.execute("SELECT * FROM products")
    products = cursor.fetchall()

    cursor.close()
    conn.close()

    html = "<h1>상품 목록</h1><ul>"

    for product in products:
        html += f"<li>{product[0]} / {product[1]} / {product[2]}원</li>"

    html += "</ul>"

    return html

if __name__ == "__main__":
    # 현재 상태
    app.run(host="0.0.0.0", port=5000)

    # [개선] X-Real-IP 출력
    # app.run(host="0.0.0.0", port=5000, request_handler=RealIPHandler)
```

10.0.12.10 대신 실제 DB-A Private IP 를 입력한다.

실행:

```
python3 app.py
```

정상 실행 예:

```
Running on http://0.0.0.0:5000
```

24-1. Flask Client IP 로그 확인

현재 상태에서는 Nginx 가 Flask 에 Reverse Proxy 로 접속하므로 Werkzeug 기본 Access Log 의 Client IP 에는 실제 외부 사용자 IP 가 아니라 WEB 서버의 Private IP 가 기록된다.

브라우저에서 WEB Public IP 에 접속하여 Flask/Werkzeug 기본 Access Log 에 WEB 서버의 Private IP 가 기록되는 것을 먼저 확인한다. 이후 app.py 의 [개선] import/class 부분의 주석을 해제하고, 현재 app.run() 한 줄은 주석 처리한 뒤 request_handler=RealIPHandler 가 포함된 [개선] app.run() 한 줄의 주석을 해제하여 다시 실행한다.

```
# 현재 상태 예시
10.0.10.x - - [08/Sep/2026 14:20:10] "GET / HTTP/1.1" 200 -
```

```
# [개선] 적용 후 예시
203.0.113.x - - [08/Sep/2026 14:21:10] "GET / HTTP/1.1" 200 -
```

※ Nginx 의 proxy_set_header X-Real-IP \$remote_addr; 설정이 실제 외부 Client IP 를 Flask 로 전달한다. RealIPHandler 는 이 헤더 값을 Werkzeug 기본 Access Log 의 Client IP 로 사용한다.

25. DB Server - MySQL

DB-A 에 MySQL 을 설치한다.

```
sudo apt update
sudo apt install -y mysql-server
```

접속:

```
sudo mysql
```

26. Database 생성

```
CREATE DATABASE testdb;

USE testdb;
```

27. products Table 생성

```
CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  price INT
);
```

28. Sample Data 입력

```
INSERT INTO products (name, price) VALUES
('노트북', 1500000),
('키보드', 100000),
('마우스', 50000),
('모니터', 300000);
```

확인:

```
SELECT * FROM products;
```

예상 결과:

id	name	price
1	노트북	1500000
2	키보드	100000
3	마우스	50000
4	모니터	300000

29. MySQL 사용자 생성

```
CREATE USER 'webuser'@'10.0.0.%'
IDENTIFIED BY 'password';

GRANT ALL PRIVILEGES ON testdb.*
TO 'webuser'@'10.0.0.%';

FLUSH PRIVILEGES;
```

30. MySQL Remote Access 설정

```
sudo vi /etc/mysql/mysql.conf.d/mysqld.cnf
```

기존:

```
bind-address = 127.0.0.1
```

변경:

```
bind-address = 0.0.0.0
```

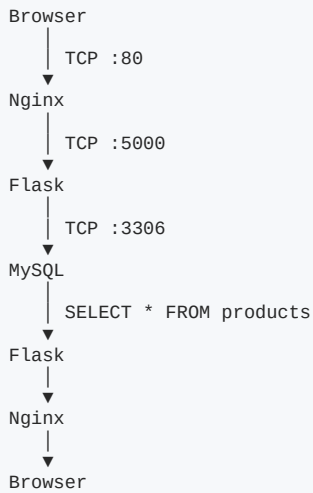
재시작:

```
sudo systemctl restart mysql
```

확인:

```
ss -ntlp | grep 3306
```

31. 최종 서비스 흐름



32. 최종 출력 예

WEB-A 또는 WEB-C 의 Public IP 에 접속한다.

```
http://WEB_PUBLIC_IP
```

브라우저 출력:

```

상품 목록
1 / 노트북 / 1500000 원
2 / 키보드 / 100000 원
3 / 마우스 / 50000 원
4 / 모니터 / 300000 원
  
```

33. 통신 검증 Mission

구축만 하는 것으로 끝나지 않는다.

실제로 허용/차단 정책이 정상적으로 동작하는지 검증한다.

필요한 경우 다음 도구를 설치한다.

```
sudo apt install -y netcat-openbsd curl
```

34. Test 1 - WEB → WAS :5000

WEB-A 에서 실행:

```
curl http://WAS_A_PRIVATE_IP:5000/
```

또는:

```
nc -zv WAS_A_PRIVATE_IP 5000
```

예상 결과

```
SUCCESS
```

WEB-C 에서도 WAS-C 를 대상으로 동일하게 검증한다.

35. Test 2 - WAS → DB :3306

WAS-A 에서 실행:

```
nc -zv DB_A_PRIVATE_IP 3306
```

WAS-C 에서도 실행:

```
nc -zv DB_A_PRIVATE_IP 3306
```

예상 결과

```
SUCCESS
```

36. Test 3 - WEB → DB :3306

WEB-A 에서 실행:

```
nc -zv -w 3 DB_A_PRIVATE_IP 3306
```

WEB-C 에서도 동일하게 실행한다.

예상 결과

FAIL

또는 Timeout.

WEB 서버가 DB 에 직접 접속할 수 있으면 Security Group 구성이 잘못된 것이다.

37. Test 4 - Bastion → WAS :22

Bastion 을 경유하여 WAS-A 에 접속한다.

```
ssh -i key.pem \  
-J ubuntu@BASTION_PUBLIC_IP \  
ubuntu@WAS_A_PRIVATE_IP
```

예상 결과

SUCCESS

38. Test 5 - Bastion → DB :22

```
ssh -i key.pem \  
-J ubuntu@BASTION_PUBLIC_IP \  
ubuntu@DB_A_PRIVATE_IP
```

예상 결과

SUCCESS

39. Test 6 - Private Server 직접 SSH

관리자 PC 에서 다음과 같은 직접 접속은 성공하면 안 된다.

```
ssh -i key.pem ubuntu@WAS_PRIVATE_IP  
ssh -i key.pem ubuntu@DB_PRIVATE_IP
```

Private EC2 에는 Public IP 자체가 없어야 한다.

예상 결과

DIRECT ACCESS IMPOSSIBLE

40. Test 7 - Internet → WEB :80

브라우저에서:

```
http://WEB_A_PUBLIC_IP
```

또는:

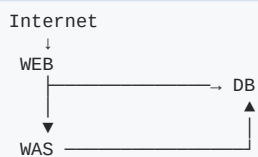
```
http://WEB_C_PUBLIC_IP
```

예상 결과

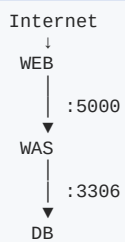
상품 목록이 정상적으로 출력된다.

41. Test 8 - WEB → DB 차단 확인이 중요한 이유

잘못된 구성:



올바른 구성:



각 Tier 는 필요한 다음 계층에만 접근하도록 제한한다.

42. 수행평가 체크리스트

A. VPC / Subnet

- ☐ VPC 10.0.0.0/16 생성
 - ☐ Availability Zone 2 개 사용
 - ☐ WEB Public Subnet 2 개
 - ☐ WAS Private Subnet 2 개
 - ☐ DB Private Subnet 2 개
 - ☐ Subnet CIDR 이 요구사항과 일치
-

B. Gateway / Routing

- ☐ Internet Gateway 생성
 - ☐ IGW 와 VPC 연결
 - ☐ NAT-GW-A 생성
 - ☐ NAT-GW-C 생성
 - ☐ NAT Gateway 각각 Elastic IP 할당
 - ☐ RT-PUBLIC 구성
 - ☐ RT-PRIVATE-A 구성
 - ☐ RT-PRIVATE-C 구성
 - ☐ Subnet Association 정확히 설정
 - ☐ AZ-A Private → NAT-GW-A
 - ☐ AZ-C Private → NAT-GW-C
-

C. EC2

- ☐ Bastion EC2 생성
 - ☐ WEB-A 생성
 - ☐ WEB-C 생성
 - ☐ WAS-A 생성
 - ☐ WAS-C 생성
 - ☐ DB-A 생성
 - ☐ Instance Type 은 t3.micro / t3.small / t3.medium 중 선택
 - ☐ WEB/Bastion Public IP 할당
 - ☐ WAS/DB Public IP 없음
-

D. Security Group

- ☐ SG-BASTION 생성
- ☐ SG-WEB 생성
- ☐ SG-WAS 생성
- ☐ SG-DB 생성
- ☐ 관리자 → Bastion :22
- ☐ 관리자 → WEB :22
- ☐ Internet → WEB :80

- ☐ Bastion → WAS :22
 - ☐ Bastion → DB :22
 - ☐ WEB → WAS :5000
 - ☐ WAS → DB :3306
 - ☐ WEB → DB :3306 Rule 없음
 - ☐ DB :3306 을 0.0.0.0/0 로 열지 않음
 - ☐ WAS :5000 을 0.0.0.0/0 로 열지 않음
-

E. Bastion

- ☐ Bastion Public IP 로 SSH 접속 성공
 - ☐ Bastion 경유 WAS-A 접속 성공
 - ☐ Bastion 경유 WAS-C 접속 성공
 - ☐ Bastion 경유 DB-A 접속 성공
 - ☐ Private EC2 직접 SSH 불가
-

F. WEB

- ☐ Nginx 설치
 - ☐ WEB-A → WAS-A Reverse Proxy 설정
 - ☐ WEB-C → WAS-C Reverse Proxy 설정
 - ☐ nginx -t 성공
 - ☐ Nginx 정상 실행
-

G. WAS

- ☐ Python 설치
 - ☐ Flask 설치
 - ☐ PyMySQL 설치
 - ☐ app.py 작성
 - ☐ Flask :5000 실행
 - ☐ DB 접속 성공
 - ☐ SELECT * FROM products 조회 성공
-

- ☐ Flask/Werkzeug 기본 Access Log 에서 WEB 서버 Private IP 확인
- ☐ [개선] RealIPHandler 적용 후 기본 Access Log 에서 실제 외부 IP 확인

H. DB

- ☐ MySQL 설치
- ☐ testdb 생성
- ☐ products Table 생성
- ☐ Sample Data 입력
- ☐ webuser 생성
- ☐ testdb.* 권한 설정
- ☐ Remote Access 설정
- ☐ WAS 에서 :3306 접속 성공

I. 통신 검증

- ☐ Internet → WEB :80 성공
- ☐ WEB → WAS :5000 성공
- ☐ WAS → DB :3306 성공
- ☐ WEB → DB :3306 실패
- ☐ Internet → WAS 직접 접근 불가
- ☐ Internet → DB 직접 접근 불가
- ☐ 관리자 → Private EC2 직접 SSH 불가
- ☐ Bastion → Private EC2 SSH 성공

43. 평가표

평가 항목	배점
VPC / Subnet 설계	10
Internet Gateway / NAT Gateway	10
Route Table 및 Subnet Association	10
EC2 배치 및 Instance Type 선택	10
Bastion Host 및 SSH 접근 구조	15
Security Group 최소 권한 구성	20
Nginx / Flask / MySQL 연동	15
허용 / 차단 통신 검증	10
총점	100

44. Security Group 평가 세부 기준

20 점

SG-BASTION - 4 점

- ☐ SSH :22 관리자 IP 만 허용
- ☐ 불필요한 Inbound Rule 없음

SG-WEB - 4 점

- ☐ HTTP :80 인터넷 허용
- ☐ SSH :22 관리자 IP 허용
- ☐ DB :3306 직접 허용 Rule 없음

SG-WAS - 6 점

- ☐ :5000 Source 가 SG-WEB

- :22 Source 가 SG-BASTION
- :5000 을 인터넷 전체에 공개하지 않음

SG-DB - 6 점

- :3306 Source 가 SG-WAS
- :22 Source 가 SG-BASTION
- :3306 을 WEB 또는 인터넷에 공개하지 않음

45. 통신 검증 평가 세부 기준

반드시 성공해야 하는 통신

```

관리자 → WEB      :22
관리자 → Bastion   :22
Bastion → WAS      :22
Bastion → DB       :22
Internet → WEB     :80
WEB → WAS          :5000
WAS → DB           :3306
  
```

반드시 실패해야 하는 통신

```

WEB → DB           :3306
Internet → WAS     :5000
Internet → DB      :3306
관리자 → WAS       :22 직접 접속
관리자 → DB        :22 직접 접속
  
```

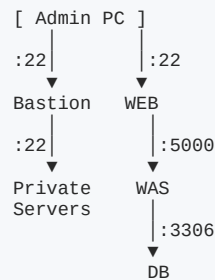
46. 최종 제출 화면

다음 결과를 확인할 수 있어야 한다.

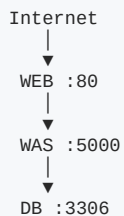
1. VPC Resource Map
2. 6 개 Subnet 구성
3. Route Table 3 개와 Subnet Association
4. NAT Gateway 2 개
5. Bastion Host
6. WEB / WAS / DB EC2 배치
7. SG-BASTION Inbound Rules
8. SG-WEB Inbound Rules
9. SG-WAS Inbound Rules
10. SG-DB Inbound Rules
11. Bastion 을 경유한 Private EC2 SSH 접속
12. WEB → WAS :5000 성공 결과
13. WAS → DB :3306 성공 결과
14. WEB → DB :3306 실패 결과
15. WEB Public IP 접속 시 상품 목록 출력 화면

47. 최종 확인

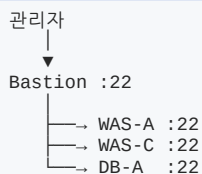
최종 아키텍처는 다음 조건을 만족해야 한다.



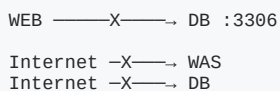
서비스 경로:



관리 경로:



차단 경로:



Mission Complete

최종 목표

단순히 서버가 동작하는 것만이 아니라,
필요한 통신은 허용되고 필요하지 않은 통신은 실제로 차단되는 3-Tier Network Security Architecture 를 완성한다.